p2 (a) In a static latency model, we assume that the load operation takes 5 cycles because it requires fetching data from memory, which involves additional time for memory access and data transfer. On the other hand, store only requires 1 cycle because it merely writes data to memory and does not involve waiting for data retrieval.

(b) A static latency model simplifies the design and analysis of systems by approximating the instruction latencies to fixed values, even though real latencies may vary. This simplification makes it easier to model and schedule instructions, while still providing reasonable performance approximations for many common scenarios.

p3 (a) There is no RAR (Read After Read) dependency because reading a value from memory or a register doesn't modify it, meaning there is no conflict in terms of data flow between two read operations. Dependencies only arise when writing to the same location.

(b) The dependency graph should be a Directed Acyclic Graph because cycles would imply circular dependencies, where one instruction depends on another in a cycle, leading to an infinite wait. A DAG ensures that there is a clear, non-circular order of execution.

p4 (a) WAR and WAW dependencies only enforce ordering because they ensure that the correct sequence of writes and reads is maintained, but they do not involve data transfer or retrieval that would introduce a timing delay.

p5 (a) The MOST_CHILD strategy prioritizes instructions with the greatest number of successors, which helps unblock a larger number of dependent instructions earlier. This increases the pool of ready instructions in subsequent cycles, providing the scheduler with more opportunities to overlap operations and improve parallelism

(b) LPT outperforms SMALLER_INDEX when there are long-latency instructions like division or multiply that are not at the end of a dependency chain.

P6 (a) The time complexity is $O(N^2)$, where N is the number of instructions. In each scheduling iteration, the naive approach scans all N nodes to check their unscheduled predecessors count to identify ready nodes. Since there are N instructions to schedule, and each iteration performs an O(N) scan, the total complexity is $O(N) \times O(N) = O(N^2)$

(b) The time complexity is O(N + E), where N is the number of nodes and E is the number of dependency edges. With memorization, each node is visited only once to compute its critical path length, and each edge in the DAG is traversed exactly once, resulting in a linear traversal of the graph

(c) In Example 1 & 2, there are strong RAW dependencies. For instance, in Example 2, each instruction strictly depends on the result of the previous one. This "chain" of dependencies forms a long Critical Path that cannot be compressed, leaving no room for the scheduler to optimize.

In Example 3, The original code suffers from Load-Use Hazards, where a register is used immediately after a lw instruction. Since lw has a 3-cycle latency, this causes significant pipeline stalls. The scheduler reorders independent lw instructions to the front, overlapping their latencies and effectively "hiding" the stall cycles.

In Example 4, The original sequence places the high latency div (10 cycles) later in the

block. The scheduler identifies div as the bottleneck of the Critical Path and moves it to Cycle 0. While the div unit is busy, the CPU executes other independent instructions, greatly increasing Instruction-Level Parallelism.

Case	Original	Scheduled	Improvement
Example 1	16	16	0%
Example 2	7	7	0%
Example 3	17	14	**17.6%**
Example 4	21	16	**23.8%**

指令调度作业核心思路

帮我查看我的分析是否正确
写一个简化版 LLVM 指令调度器
核心目标：

输入：RISC-V 指令序列...

你的分析非常出色，逻辑清晰且完全抓住了实验指导书的核心要点。你不仅理清了开发顺序，还准确地识别了最容易出错的“坑”（如依赖释放时机和优先级计算）。

以下是对你分析的进一步**精细化补充**和**潜在风险提示**，帮助你确保拿到满分：

---

1. 优先级计算的细节修正 (Critical Path)

你在 DFS 伪代码中有一个细节需要注意。根据讲义公式 (1)：

- **对于 RAW 后继**：优先级等于 `当前节点延迟 + 后继节点优先级`。
- **对于 WAR/WAW 后继**：优先级直接等于 `后继节点优先级`（不加当前节点延迟）。
- **记忆化**：务必使用一个数组存储已计算过的 `priority`，否则在处理大型 DAG 时复杂度会爆炸。

2. 调度器逻辑的“释放时机” (Schedule)

这是整个 Project 3 最核心的逻辑。请务必在代码中体现以下逻辑差异：

- **Issue 时 (Cycle C)**：
  - 将 `best_node` 放入 `inflight` 队列，记录其完成周期为 $C + \text{latency}$。

2. 调度器逻辑的“释放时机” (Schedule)

这是整个 Project 3 最核心的逻辑 。请务必在代码中体现以下逻辑差异：

- **Issue 时 (Cycle C)：**
  - 将 `best_node` 放入 `inflight` 队列，记录其完成周期为 $C + \text{latency}$ 。 +1
  - **立即遍历** `best_node` 的 **WAR** 和 **WAW** 后继，减少它们的前驱计数 。如果计数归零，加入 `ready` 集合 。 +4
- **Retire 时 (Cycle C 到达完成周期)：**
  - 从 `inflight` 移除已完成节点 。 +1
  - **此时才遍历**该节点的 **RAW** 后继，减少前驱计数并更新 `ready` 集合 。 +2

3. Tie-breaking 的隐藏规则

在 `selectBestNode()` 中，老师特别提到了一个嵌套规则 ：

> "If two candidates still tie on MOST_CHILD or LPT, use SMALLER_INDEX to break the tie."

这意味着你的比较逻辑应该是：

1. 比较 `priority`。
2. 若相同，比较 `MOST_CHILD`。
3. 若 `MOST_CHILD` 也相同，比较 `index`（越小越优）。
4. （对 LPT 同理，最后都要回退到 `index` 比较） 。